
Python pour économistes

Livret d'exercices appliqués avec Our World in Data

Université de Lille – L2 Économie

Année 2026–2027

Ce livret rassemble les cinq TPs. Chaque séance articule apprentissage de Python, statistiques descriptives simples, pseudo-code et applications économiques fondées sur des données Our World in Data.

Obligatoire

exercices à rendre et résultats attendus.

Pseudo-code

étapes à compléter avant d'écrire le Python.

Pathways

chemins de fichiers, dossiers courants et sorties reproductibles.

Compléments

entraînement supplémentaire pour consolider la séance.

Objectif : apprendre Python en transformant progressivement des indicateurs économiques OWID en scripts clairs, graphiques et interprétations prudentes.

Mode d'emploi du livret

Comment travailler une séance

1. Lire la question fil rouge, la source OWID et les livrables avant de coder.
2. Compléter le pseudo-code : données, étapes Python, sortie statistique, contrôle.
3. Écrire le script par petits blocs et l'exécuter souvent.
4. Vérifier les chemins de fichiers, surtout le dossier courant et le dossier `out`.
5. Terminer par une interprétation courte, prudente et reliée aux résultats économiques.

Table des matières

Mode d'emploi du livret	1
TP1 – Premiers scripts et statistiques descriptives simples	2
TP2 – Boucles, conditions et proportions avec données OWID	5
TP3 – Fonctions et comparaison de groupes	8
TP4 – Graphiques et simulation simple à partir d OWID	11
TP5 – Mini-projet pandas avec Our World in Data	15

TP1 – Premiers scripts et statistiques descriptives simples

Python pour économistes – Université de Lille, L2 Économie

Question fil rouge

Comment écrire un premier script Python pour décrire quelques indicateurs économiques et sociaux issus d'Our World in Data ?

Livrables

Un fichier `tp1_nom_prenom.py` qui s'exécute sans erreur, affiche des statistiques simples et contient trois phrases d'interprétation.

Méthode de travail

Lire	identifier les données, la question et le livrable attendu.
Décomposer	écrire les étapes en pseudo-code avant de coder.
Exécuter	lancer le script complet depuis le bon dossier.
Vérifier	contrôler les sorties, les fichiers créés et l'interprétation.

Parcours obligatoire

Les exercices 1 à 7 sont obligatoires. Le but principal est Python : variables, types, listes, calculs, affichage lisible et commentaires. La statistique sert seulement de support.

Source des données

Les valeurs utilisées dans ce TP sont des extraits pédagogiques arrondis de graphiques Our World in Data. Pour l'instant, elles sont écrites directement dans le script afin de ne pas dépendre de pandas.

```
https://ourworldindata.org/grapher/life-expectancy.csv  
https://ourworldindata.org/grapher/population.csv
```

Logique 1 – Décrire une petite liste d'indicateurs

PSEUDO-CODE

Compléter la logique avant d'écrire le code.

```
# Objectif : decrire une liste de valeurs OWID  
# Entree : une liste de pays et une liste de valeurs  
# 1. Afficher les donnees de depart  
# 2. Calculer le nombre d'observations  
# 3. Calculer la moyenne  
# 4. Reperer la valeur minimale et maximale  
# 5. Afficher une phrase avec une unite  
# Sortie : statistiques + interpretation courte
```

A. Dossier, script, exécution

Exercice 1 – Préparer un dossier de travail

OBLIGATOIRE

Créer un dossier TP1 contenant un script `tp1_nom_prenom.py`. Depuis le terminal, vérifier le dossier courant puis exécuter le script.

```
mkdir TP1
cd TP1
pwd
python3 tp1_nom_prenom.py
```

Dans le script, ajouter un commentaire qui explique la différence entre le dossier courant et le fichier exécuté.

Exercice 2 – Premières variables économiques

OBLIGATOIRE

Dans le script, créer les variables suivantes puis afficher leur valeur et leur type.

```
pays = "France"
annee = 2021
esperance_vie = 82.5
population_millions = 67.7
```

Afficher ensuite une phrase lisible avec une f-string :

```
print(f"En {annee}, {pays} compte environ {population_millions} millions d'habitants.")
```

B. Listes et statistiques simples

Exercice 3 – Créer un extrait OWID simplifié

OBLIGATOIRE

Créer deux listes alignées :

```
pays = ["France", "Germany", "Spain", "Italy", "Poland", "Sweden"]
esperance_vie = [82.5, 80.6, 83.0, 82.9, 76.5, 83.1]
```

Les valeurs sont arrondies et exprimées en années. Afficher la première valeur, la dernière valeur et le nombre de pays.

Exercice 4 – Moyenne, minimum, maximum

OBLIGATOIRE

À partir de la liste `esperance_vie`, calculer :

- a. le nombre d'observations avec `len(...)`;
- b. la moyenne avec `sum(...) / len(...)`;
- c. le minimum avec `min(...)`;
- d. le maximum avec `max(...)`.

Afficher chaque résultat avec une unité claire : années.

Exercice 5 – Pourcentage d'évolution**OBLIGATOIRE**

Pour la France, on donne un extrait simplifié de l'espérance de vie :

```
france_2000 = 79.1  
france_2021 = 82.5
```

Calculer la variation en années, puis le taux d'évolution en pourcentage :

$$\frac{\text{valeur finale} - \text{valeur initiale}}{\text{valeur initiale}} \times 100.$$

Afficher le résultat avec deux décimales.

C. Interpréter sans surcharger

Exercice 6 – Phrase d'interprétation**OBLIGATOIRE**

Ajouter trois commentaires à la fin du script :

- a. une phrase qui donne la moyenne de l'extrait ;
- b. une phrase qui compare la France à cette moyenne ;
- c. une limite : pourquoi six pays ne suffisent pas à décrire le monde ?

Exercice 7 – Contrôle final du script**OBLIGATOIRE**

Le script rendu doit contenir au minimum : une variable `str`, une variable `int`, une variable `float`, une liste, une moyenne, un pourcentage d'évolution, une f-string et trois phrases d'interprétation.

D. Entraînement facultatif

Extension 1 – Dix petits entraînements OWID

Choisir au moins deux questions si le parcours obligatoire est terminé.

- a. Ajouter "Japan" et une valeur d'espérance de vie dans les deux listes.
- b. Créer une liste de populations en millions pour les mêmes pays.
- c. Calculer la population totale de l'extrait.
- d. Calculer la part de la France dans cette population totale.
- e. Afficher `pays[0]` et `esperance_vie[0]` dans une même phrase.
- f. Tester `esperance_vie[0] > 80` et interpréter le booléen obtenu.
- g. Remplacer une valeur par une valeur manquante fictive `0`, puis expliquer pourquoi cela fausse la moyenne.
- h. Créer une variable `source = "Our World in Data"` et l'afficher dans les résultats.
- i. Arrondir la moyenne avec `round(moyenne, 1)`.
- j. Écrire une phrase expliquant pourquoi les unités sont importantes.

TP2 – Boucles, conditions et proportions avec données OWID

Python pour économistes – Université de Lille, L2 Économie

Question fil rouge

Comment utiliser des boucles et des conditions pour classer des observations économiques simples ?

Livrables

Un fichier `tp2_nom_prenom.py` contenant une boucle, une condition, une proportion et une interprétation économique courte.

Méthode de travail

Lire	identifier les données, la question et le livrable attendu.
Décomposer	écrire les étapes en pseudo-code avant de coder.
Exécuter	lancer le script complet depuis le bon dossier.
Vérifier	contrôler les sorties, les fichiers créés et l'interprétation.

Parcours obligatoire

Les exercices 1 à 7 sont obligatoires. La notion statistique principale est la proportion. Les seuils utilisés sont pédagogiques : ils servent à apprendre `if`, `for` et les compteurs.

Source des données

Les valeurs de CO₂ par habitant sont des extraits pédagogiques arrondis issus d'Our World in Data.

<https://ourworldindata.org/grapher/co-emissions-per-capita.csv>

Logique 1 – Classer des observations

PSEUDO-CODE

```
# Objectif : classer des pays selon un indicateur
# Entree : liste de pays et valeurs de CO2 par habitant
# Pour chaque pays :
#     lire la valeur
#     comparer la valeur a un seuil
#     afficher une categorie
# Compter combien de pays sont dans chaque categorie
# Sortie : effectifs, proportions, interpretation
```

A. Représenter les données

Exercice 1 – Créer une liste de dictionnaires**OBLIGATOIRE**

Créer la liste suivante. Chaque dictionnaire représente une observation.

```
donnees = [  
    {"pays": "France", "co2_pc": 4.7},  
    {"pays": "Germany", "co2_pc": 7.9},  
    {"pays": "Spain", "co2_pc": 5.0},  
    {"pays": "Poland", "co2_pc": 8.4},  
    {"pays": "United States", "co2_pc": 14.9},  
    {"pays": "India", "co2_pc": 2.0},  
    {"pays": "China", "co2_pc": 8.0},  
    {"pays": "World", "co2_pc": 4.7}  
]
```

Afficher le nombre d'observations et la première observation.

Exercice 2 – Parcourir les observations**OBLIGATOIRE**

Avec une boucle for, afficher une phrase par pays :

```
for obs in donnees:  
    print(f"{obs['pays']} : {obs['co2_pc']} tonnes par habitant")
```

Modifier ensuite l'affichage pour arrondir à une décimale.

B. Conditions et proportions

Exercice 3 – Classifier avec une condition**OBLIGATOIRE**

On utilise trois catégories pédagogiques :

- faible si `co2_pc < 5`;
- intermediaire si `5 <= co2_pc < 10`;
- eleve si `co2_pc >= 10`.

Écrire une boucle qui affiche la catégorie de chaque pays.

Exercice 4 – Compter une proportion**OBLIGATOIRE**

Compter le nombre de pays dont les émissions dépassent 5 tonnes par habitant. Calculer ensuite la proportion :

$$\frac{\text{nombre de pays au-dessus du seuil}}{\text{nombre total de pays}} \times 100.$$

Afficher une phrase du type : 62.5 % des observations sont au-dessus du seuil.

Exercice 5 – Créer une fonction de classement**OBLIGATOIRE**

Écrire une fonction `classe_co2(valeur)` qui renvoie "faible", "intermediaire" ou "eleve". L'utiliser dans une boucle pour afficher le classement de tous les pays.

C. Interpréter

Exercice 6 – Question économique**OBLIGATOIRE**

Ajouter quatre commentaires à la fin du script :

- a. quel pays de l'extrait a la valeur la plus élevée ?
- b. la France est-elle au-dessus ou au-dessous de la valeur mondiale ?
- c. quelle proportion de l'extrait dépasse 5 tonnes par habitant ?
- d. pourquoi ne faut-il pas confondre CO₂ par habitant et CO₂ total ?

Exercice 7 – Contrôle final**OBLIGATOIRE**

Le script doit contenir au minimum : une liste de dictionnaires, une boucle `for`, une condition `if/elif/else`, un compteur, une proportion, une fonction et une interprétation.

D. Entraînement facultatif**Extension 1 – Dix petits entraînements**

Choisir au moins deux questions si le parcours obligatoire est terminé.

- a. Changer le seuil de 5 à 7 tonnes et recalculer la proportion.
- b. Créer une liste contenant seulement les pays classés `eleve`.
- c. Calculer la moyenne de `co2_pc` avec une boucle, sans `sum()`.
- d. Ajouter un pays et vérifier que le code fonctionne encore.
- e. Afficher les pays au-dessous de la valeur mondiale.
- f. Compter séparément les pays `faible`, `intermediaire` et `eleve`.
- g. Écrire une fonction `au-dessus_seuil(valeur, seuil)`.
- h. Trier manuellement deux pays : lequel a la valeur la plus élevée ?
 - i. Écrire une phrase sur la limite d'un classement par seuil.
 - j. Expliquer pourquoi une proportion dépend de l'échantillon choisi.

TP3 – Fonctions et comparaison de groupes

Python pour économistes – Université de Lille, L2 Économie

Question fil rouge

Comment écrire des fonctions Python pour comparer des groupes de pays à partir d'indicateurs OWID ?

Livrables

Un fichier `tp3_nom_prenom.py`, un tableau texte de statistiques descriptives simples et une interprétation de cinq lignes.

Méthode de travail

Lire	identifier les données, la question et le livrable attendu.
Décomposer	écrire les étapes en pseudo-code avant de coder.
Exécuter	lancer le script complet depuis le bon dossier.
Vérifier	contrôler les sorties, les fichiers créés et l'interprétation.

Parcours obligatoire

Les exercices 1 à 7 sont obligatoires. Les notions statistiques obligatoires sont : moyenne, médiane, minimum, maximum et proportion. Les quartiles et l'écart-type restent facultatifs.

Source des données

Les valeurs de PIB par habitant sont des extraits pédagogiques arrondis d'Our World in Data.

<https://ourworldindata.org/grapher/gdp-per-capita-worldbank.csv>

Logique 1 – Automatiser une statistique

PSEUDO-CODE

```
# Objectif : comparer deux listes de valeurs
# Entrees : liste_groupe_a, liste_groupe_b
# Définir une fonction moyenne(liste)
# Définir une fonction mediane(liste)
# Appliquer les fonctions aux deux groupes
# Comparer les resultats
# Sortie : tableau + phrase d'interpretation
```

A. Données et rappels

Exercice 1 – Créer deux groupes de pays**OBLIGATOIRE**

Créer deux listes de PIB par habitant, exprimées en milliers de dollars internationaux et arrondies pour l'exercice.

```
groupe_europe = [46.0, 54.0, 34.0, 38.0, 22.0, 61.0]
groupe_monde = [46.0, 76.0, 19.0, 10.0, 7.0, 3.0]
```

Ajouter deux listes de noms pour documenter les valeurs :

```
pays_europe = ["France", "Germany", "Spain", "Italy", "Poland", "Sweden"]
pays_monde = ["France", "United States", "China", "Brazil", "India", "Ethiopia"]
```

Exercice 2 – Rappel intuitif**OBLIGATOIRE**

Ajouter dans le script trois commentaires :

- la moyenne résume un niveau général ;
- la médiane est la valeur centrale après tri ;
- min et max donnent l'étendue des valeurs observées.

B. Fonctions**Exercice 3 – Fonction moyenne****OBLIGATOIRE**

Écrire une fonction `moyenne(valeurs)` qui renvoie la moyenne d'une liste. La tester sur les deux groupes.

```
def moyenne(valeurs):
    return sum(valeurs) / len(valeurs)
```

Afficher les résultats avec une unité : milliers de dollars par habitant.

Exercice 4 – Fonction médiane guidée**OBLIGATOIRE**

Pour une liste de longueur paire, on peut prendre la moyenne des deux valeurs centrales. Compléter la fonction :

```
def mediane_paire(valeurs):
    valeurs_triees = sorted(valeurs)
    n = len(valeurs_triees)
    milieu_droit = n // 2
    milieu_gauche = milieu_droit - 1
    return (valeurs_triees[milieu_gauche] + valeurs_triees[milieu_droit]) / 2
```

Tester la fonction sur les deux groupes.

Exercice 5 – Min, max et proportion**OBLIGATOIRE**

Pour chaque groupe, calculer :

- la valeur minimale ;
- la valeur maximale ;
- la proportion de pays au-dessus de 30 milliers de dollars par habitant.

Conseil : utiliser un compteur et une boucle.

C. Comparer et interpréter

Exercice 6 – Tableau texte

OBLIGATOIRE

Afficher un petit tableau texte avec les colonnes : `groupe`, `moyenne`, `mediane`, `min`, `max`, `part_au_dessus_30`. L'objectif est la lisibilité, pas la perfection typographique.

Exercice 7 – Question économique

OBLIGATOIRE

Ajouter cinq commentaires à la fin du script :

- a. quel groupe a la moyenne la plus élevée ?
- b. quel groupe a la médiane la plus élevée ?
- c. pourquoi moyenne et médiane peuvent-elles raconter deux choses différentes ?
- d. pourquoi le choix des pays influence-t-il fortement le résultat ?
- e. quelle limite voyez-vous à travailler sur seulement six pays par groupe ?

D. Entraînement facultatif

Extension 1 – Statistiques supplémentaires sans pression

Choisir au moins deux questions si le parcours obligatoire est terminé.

- a. Ajouter un pays dans chaque groupe et vérifier que les fonctions marchent encore.
- b. Calculer l'étendue : `max - min`.
- c. Écrire une fonction `proportion_au_dessus(valeurs, seuil)`.
- d. Tester le seuil de 50 au lieu de 30.
- e. Écrire une fonction `resume(valeurs)` qui affiche moyenne, médiane, min et max.
- f. Comparer la moyenne avant et après ajout d'une valeur très élevée.
- g. Calculer un premier quartile avec une méthode simple de votre choix.
- h. Calculer l'écart-type à partir d'une formule fournie par l'enseignant.
- i. Écrire une phrase expliquant pourquoi l'écart-type mesure la dispersion.
- j. Transformer les valeurs en dollars au lieu de milliers de dollars.

TP4 – Graphiques et simulation simple à partir d'OWID

Python pour économistes – Université de Lille, L2 Économie

Question fil rouge

Comment utiliser Python pour visualiser une distribution et comprendre intuitivement la stabilisation d'une moyenne ?

Livrables

Un fichier `tp4_nom_prenom.py`, deux figures `.png` et une interprétation courte.

Méthode de travail

Lire	identifier les données, la question et le livrable attendu.
Décomposer	écrire les étapes en pseudo-code avant de coder.
Exécuter	lancer le script complet depuis le bon dossier.
Vérifier	contrôler les sorties, les fichiers créés et l'interprétation.

Parcours obligatoire

Les exercices 1 à 8 sont obligatoires. La simulation est volontairement simple : elle sert à apprendre les boucles, `random.choice` et les graphiques, pas à faire de statistique avancée.

Source des données

Les taux de fécondité sont des extraits pédagogiques arrondis d'Our World in Data.

<https://ourworldindata.org/grapher/children-per-woman-un.csv>

Logique 1 – Visualiser puis simuler

PSEUDO-CODE

```
# Objectif : observer une distribution puis simuler des tirages
# Entree : liste de taux de fecondite
# 1. Calculer la moyenne observee
# 2. Tracer un graphique en barres
# 3. Tirer au hasard une valeur dans la liste
# 4. Repeter le tirage plusieurs fois
# 5. Calculer la moyenne des tirages
# 6. Observer si la moyenne se stabilise quand n augmente
```

A. Préparer les données et un graphique

Exercice 1 – Créer l'extrait OWID**OBLIGATOIRE**

Créer un dictionnaire où les clés sont des pays ou régions et les valeurs des taux de fécondité arrondis.

```
fertilite = {  
    "World": 2.3,  
    "France": 1.8,  
    "United States": 1.6,  
    "Japan": 1.2,  
    "India": 2.0,  
    "Nigeria": 4.5,  
    "Niger": 6.1,  
    "Brazil": 1.6,  
    "Ethiopia": 3.9,  
    "China": 1.0  
}
```

Afficher le nombre d'observations et la moyenne simple de ces valeurs.

Exercice 2 – Graphique en barres**OBLIGATOIRE**

Avec `matplotlib`, tracer un graphique en barres des taux de fécondité.

```
import matplotlib.pyplot as plt  
  
pays = list(fertilite.keys())  
valeurs = list(fertilite.values())  
  
plt.bar(pays, valeurs)  
plt.xticks(rotation=45, ha="right")  
plt.ylabel("Enfants par femme")  
plt.title("Taux de fecondite - extrait OWID")  
plt.tight_layout()  
plt.savefig("fertilite_barres.png", dpi=300)
```

Exercice 3 – Interpréter le graphique**OBLIGATOIRE**

Ajouter trois commentaires :

- quel pays a le taux le plus élevé dans l'extrait ?
- quel pays a le taux le plus faible ?
- pourquoi ce graphique est-il plus lisible qu'une liste brute ?

B. Simulation très guidée**Exercice 4 – Un tirage aléatoire****OBLIGATOIRE**

Importer `random`, puis tirer une valeur au hasard dans la liste des taux.

```
import random  
  
tirage = random.choice(valeurs)  
print(tirage)
```

Exécuter plusieurs fois le script : le résultat est-il toujours le même ?

Exercice 5 – Moyenne de plusieurs tirages

OBLIGATOIRE

Compléter le code suivant pour calculer la moyenne de 20 tirages.

```
tirages = []
for i in range(20):
    tirages.append(random.choice(valeurs))

moyenne_tirages = sum(tirages) / len(tirages)
print(moyenne_tirages)
```

Comparer cette moyenne à la moyenne de la liste initiale.

Exercice 6 – Stabilisation de la moyenne

OBLIGATOIRE

Calculer la moyenne des tirages pour plusieurs tailles : 5, 20, 100, 1000. Afficher les résultats dans une phrase.

```
tailles = [5, 20, 100, 1000]
for n in tailles:
    tirages = []
    for i in range(n):
        tirages.append(random.choice(valeurs))
    print(n, sum(tirages) / len(tirages))
```

C. Visualiser la simulation

Exercice 7 – Graphique de stabilisation

OBLIGATOIRE

Créer deux listes : `tailles` et `moyennes_simulees`. Tracer `moyennes_simulees` en fonction de `tailles`. Ajouter une ligne horizontale correspondant à la moyenne de la liste initiale avec `plt.axhline(...)`. Sauvegarder `simulation_moyenne.png`.

Exercice 8 – Interprétation économique

OBLIGATOIRE

Ajouter quatre commentaires :

- que représente la moyenne de l'extrait ?
- que montre la simulation quand le nombre de tirages augmente ?
- pourquoi ce résultat ne décrit-il pas directement la population mondiale ?
- pourquoi la sélection des pays est-elle importante ?

D. Entraînement facultatif

Extension 1 – Pour aller plus loin, sans obligation

Choisir au moins deux questions si le parcours obligatoire est terminé.

- a. Fixer la graine avec `random.seed(123)` et observer l'effet.
- b. Faire 10 simulations de taille 20 et comparer les moyennes obtenues.
- c. Tracer un histogramme des 1000 tirages.
- d. Supprimer Niger de la liste et recalculer la moyenne.
- e. Ajouter une ligne pour la médiane de la liste initiale.
- f. Écrire une fonction `moyenne_tirages(n)`.
- g. Tester des tailles 10, 50, 500, 5000.
- h. Calculer l'écart entre moyenne simulée et moyenne initiale.
- i. Lire intuitivement la loi des grands nombres : que veut dire “stabiliser” ?
- j. Extension avancée : construire un intervalle `moyenne +/- 2 * ecart_type / racine(n)` avec une formule donnée.

TP5 – Mini-projet pandas avec Our World in Data

Python pour économistes – Université de Lille, L2 Économie

Question fil rouge

Comment importer un CSV OWID, produire des statistiques descriptives simples, tracer un graphique et rédiger une interprétation économique ?

Livrables

Un fichier `tp5_nom_prenom.py`, une figure `out/gdp_top10.png`, un fichier `out/gdp_selection.csv` et une interprétation de six à huit lignes.

Méthode de travail

Lire	identifier les données, la question et le livrable attendu.
Décomposer	écrire les étapes en pseudo-code avant de coder.
Exécuter	lancer le script complet depuis le bon dossier.
Vérifier	contrôler les sorties, les fichiers créés et l'interprétation.

Parcours obligatoire

Les exercices 1 à 9 sont obligatoires. Ce TP est le mini-projet final : il combine chemins de fichiers, `pandas`, statistiques descriptives simples, graphique et interprétation.

Pathways : travailler avec un CSV OWID

Le TP lit directement un CSV publié par Our World in Data. Si la connexion internet ne fonctionne pas en salle, télécharger le CSV avec un navigateur et placer le fichier dans un dossier `data`.

<https://ourworldindata.org/grapher/gdp-per-capita-worldbank.csv>

Logique 1 – Pipeline pandas final

PSEUDO-CODE

```
# Objectif : mini-projet descriptif avec un CSV OWID
# 1. Lire le CSV
# 2. Inspecter lignes, colonnes et types
# 3. Renommer les colonnes utiles
# 4. Filtrer une année et quelques pays
# 5. Calculer moyenne, médiane, min, max
# 6. Tracer un graphique lisible
# 7. Exporter un tableau propre
# 8. Rédiger une interprétation prudente
```

A. Importer et inspecter

Exercice 1 – Préparer le script**OBLIGATOIRE**

Créer l'arborescence suivante :

```
TP5/  
  tp5_nom_prenom.py  
  out/
```

Dans le script, créer le dossier out.

```
from pathlib import Path  
  
out_dir = Path("out")  
out_dir.mkdir(exist_ok=True)
```

Exercice 2 – Lire le CSV OWID**OBLIGATOIRE**

Lire le fichier depuis l'URL OWID.

```
import pandas as pd  
  
url = "https://ourworldindata.org/grapher/gdp-per-capita-worldbank.csv"  
df = pd.read_csv(url)  
  
print(df.head())  
print(df.shape)  
print(df.columns)
```

Ajouter un commentaire indiquant le nombre de lignes et de colonnes.

Exercice 3 – Renommer proprement**OBLIGATOIRE**

Le nom de la colonne de valeur peut être long. On le récupère automatiquement.

```
value_col = df.columns[-1]  
  
df = df.rename(columns={  
    "Entity": "country",  
    "Code": "code",  
    "Year": "year",  
    value_col: "gdp_pc"  
})
```

Afficher `df[["country", "year", "gdp_pc"]].head()`.

B. Filtrer et décrire**Exercice 4 – Sélectionner une année récente****OBLIGATOIRE**

Créer un tableau `latest` avec la dernière année disponible dans le fichier, puis retirer les valeurs manquantes.

```
latest_year = df["year"].max()  
latest = df.loc[df["year"] == latest_year].copy()  
latest = latest.dropna(subset=["gdp_pc"])  
print(latest_year)
```

```
print(latest.shape)
```

Exercice 5 – Sélectionner quelques pays

OBLIGATOIRE

Créer une liste de pays et filtrer le tableau.

```
pays_selection = [
    "France", "Germany", "Spain", "Poland",
    "United States", "China", "India", "Brazil"
]
selection = latest.loc[latest["country"].isin(pays_selection)].copy()
print(selection[["country", "gdp_pc"]])
```

Exercice 6 – Statistiques descriptives simples

OBLIGATOIRE

Sur selection, calculer :

- a. moyenne ;
- b. médiane ;
- c. minimum ;
- d. maximum ;
- e. proportion de pays au-dessus de la moyenne de la sélection.

On peut utiliser `mean()`, `median()`, `min()` et `max()`.

C. Graphique et export

Exercice 7 – Graphique du top 10

OBLIGATOIRE

Créer un graphique en barres des dix pays les plus élevés dans latest.

```
import matplotlib.pyplot as plt

top10 = latest.sort_values("gdp_pc", ascending=False).head(10)
ax = top10.plot(kind="bar", x="country", y="gdp_pc", legend=False)
ax.set_xlabel("Pays")
ax.set_ylabel("PIB par habitant")
ax.set_title(f"Top 10 du PIB par habitant - {latest_year}")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(out_dir / "gdp_top10.png", dpi=300)
```

Exercice 8 – Exporter un tableau propre

OBLIGATOIRE

Exporter selection dans out/gdp_selection.csv avec seulement country, code, year, gdp_pc.

```
selection[["country", "code", "year", "gdp_pc"]].to_csv(
    out_dir / "gdp_selection.csv",
    index=False
)
```

Exercice 9 – Interprétation finale**OBLIGATOIRE**

Ajouter six à huit lignes de commentaire :

- a. année étudiée ;
- b. pays le plus élevé dans la sélection ;
- c. pays le plus faible dans la sélection ;
- d. moyenne et médiane de la sélection ;
- e. comparaison de la France à la moyenne ;
- f. limite : choix des pays ;
- g. limite : le PIB par habitant ne mesure pas toutes les dimensions du développement.

D. Extension facultative**Extension 1 – Pour les étudiants plus rapides**

Choisir au moins deux questions si le mini-projet est terminé.

- a. Tracer uniquement les pays de la sélection.
- b. Comparer la France à l'Allemagne, aux États-Unis et à la Chine.
- c. Créer une colonne `above_selection_mean`.
- d. Exporter le `top10` dans `out/gdp_top10.csv`.
- e. Tracer l'évolution du PIB par habitant de la France depuis 1990.
- f. Ajouter le monde "World" si présent dans le fichier.
- g. Remplacer `latest_year` par une année choisie manuellement.
- h. Calculer le rapport entre le maximum et le minimum de la sélection.
- i. Importer aussi l'espérance de vie depuis <https://ourworldindata.org/grapher/life-expectancy.csv>.
- j. Extension avancée : fusionner PIB par habitant et espérance de vie pour une même année.